Lab 3D: Capstone - Multi-Agent Build + Headless CI Check

Duration: 35 min

After: Scale, Cost & New Surfaces lecture (Day 3, Block 4)

Prerequisites: Labs 3A-3C complete (MCP server, subagents, ci-review script)

Objective

Run everything at once: a multi-agent build of a real feature on your toolkit repo, graded against a rubric, validated by your headless CI check. Success is binary: **toolkit repo complete + CI run green.**

Deliverable (toolkit chain 12 of 12 - FINAL)

The finished personal toolkit repo containing all five artifacts - **CLAUDE.md, skill, hook, command, plugin manifest** - plus a new feature built by agents and a green CI run proving it.

> ***START MARKER:*** `cd business-dashboard`*, terminal open.*

Fell behind? Minimum viable start:

```
Set up this project with: CLAUDE.md conventions, an Express server with src/routes/, a
code-review skill in .claude/skills/, a PostToolUse prettier hook in .claude/settings.json,
a /review command, subagents security-auditor and grader in .claude/agents/, and
scripts/ci-review.sh that runs claude -p with --allowedTools "Read,Grep,Glob" --max-turns 10
and fails below score 6.
```

Phase 1: Toolkit Inventory Gate (5 min)

Before building, verify all five toolkit artifacts exist. Start `claude` and run:

```
Verify this repo contains all five toolkit artifacts and report PRESENT/MISSING for each:
1. CLAUDE.md (root, under 200 lines)
2. A skill: .claude/skills/code-review/SKILL.md with
name/description/disable-model-invocation/allowed-tools frontmatter
3. A hook: .claude/hooks/post-edit-format.sh registered as PostToolUse in
.claude/settings.json
4. A command: .claude/commands/review.md using $ARGUMENTS
5. A plugin manifest: ../dashboard-toolkit-plugin/.claude-plugin/plugin.json (or plugin
installed from team-marketplace)
```

Expected output marker: five PRESENT lines. Fix any MISSING with the relevant lab's recovery prompt before continuing - the capstone grade depends on it.

Phase 2: Plan the Feature (5 min)

Shift+Tab to **Plan mode**, then:

```
Plan a "Weekly Report" feature:
- GET /api/report/weekly - JSON summary: metrics by category, task completion rate, top 3
high-priority open tasks
- A "Weekly Report" section on the dashboard UI rendering that data
- Data shapes should match the dashboard-metrics MCP server where relevant
- Must follow CLAUDE.md conventions and pass our rubric.md criteria
Produce a step-by-step plan with file list and test strategy. Save it to
IMPLEMENTATION_PLAN.md when I approve.
```

Review, iterate once if needed, approve, and let it save the plan.

Expected output marker: `IMPLEMENTATION_PLAN.md` exists; no source files changed yet (Plan mode held the line).

Phase 3: Multi-Agent Build (12 min)

Shift+Tab back to Default (or Auto if your Lab 1D permission rules are in place), then orchestrate:

```
Execute IMPLEMENTATION_PLAN.md as a multi-agent build:
1. Spawn a builder subagent to implement the API endpoint and UI section per the plan
2. In parallel, spawn a test-writer subagent to write tests for the endpoint (happy path,
empty data, error handling)
3. When both finish, run the security-auditor subagent over the changed files
4. Finally, have the grader subagent score the implementation against rubric.md
Report each agent's summary and the grader's verdict.
```

Watch: builder and test-writer run as background subagents in parallel (streaming since week 27); auditor and grader run as review passes.

Expected output marker: all four agent summaries + a grader table ending `TOTAL: n/10`. If the verdict is REWORK:

```
Fix only the criteria scored below 2, then have the grader re-score.
```

Expected output marker: PASS (>= 8/10).

Verify by hand - open http://localhost:3100 and confirm the Weekly Report section renders. Then:

```
/review src/routes/
```

Commit:

```
Commit the weekly report feature with a conventional message, mentioning it was built by
parallel subagents and graded at [score].
```

Phase 4: Headless CI Check - the Green Light (8 min)

Exit Claude (`exit`). Run your Lab 3C gate against the finished repo:

```
./scripts/ci-review.sh
```

Expected output marker:

```
Review score: 8
Cost: $0.0x
REVIEW PASSED
```

Exit code 0 = **CI green.** If it fails, read the findings:

```
jq -r '.result' review.json
```

Start `claude`, paste the top issues with "Fix these findings, then re-run scripts/ci-review.sh", and repeat until green.

Have GitHub + Lab 3C's workflow? Push a PR instead and let `anthropics/claude-code-action@v1` post the advisory review - same gate, running in the cloud:

```
git checkout -b feature/weekly-report && git push -u origin feature/weekly-report
gh pr create --title "feat: weekly report (capstone)" --body "Multi-agent build,
grader-verified, CI-checked"
gh run watch
```

Phase 5: Retrospective (5 min)

```
Using the git history and current codebase, summarize the evolution of this repo across all
12 labs: what each toolkit artifact adds, how the agents divided today's work, and the total
cost of this session (/cost data). Keep it under 300 words - this is my week-one plan seed.
```

Count your prompts across three days. A few dozen messages shipped: a working system, a policy file, a tested skill, enforced hooks, a parameterized command, a distributable plugin, an MCP server, a graded multi-agent feature, and a CI pipeline. That's the leverage.

FINISH MARKER - Capstone Success Criteria (binary)

Complete = BOTH true:

- [ ] **Toolkit repo complete:** CLAUDE.md + skill + hook + command + plugin manifest all PRESENT (Phase 1 gate)
- [ ] **CI run green:** `scripts/ci-review.sh` exits 0 (or the GitHub Action run completes with the advisory review posted)

Supporting evidence:

- [ ] IMPLEMENTATION_PLAN.md produced in Plan mode
- [ ] Feature built by parallel subagents, not the main session
- [ ] Grader verdict PASS (>= 8/10) with evidence table
- [ ] Weekly Report renders in the browser
- [ ] Conventional commit history tells the story

If Stuck

Problem	Recovery
Phase 1 shows MISSING artifacts	Run that lab's recovery prompt - do not skip the gate
Agents collide on the same file	Re-run with explicit ownership ("builder owns src/, test-writer owns tests/ only")
Grader keeps failing one criterion	Read its evidence column - fix that exact line, don't rewrite the feature
ci-review.sh score seems wrong	`jq -r '.result' review.json` and read the findings - the score follows them
Ran out of time	Binary criteria first: inventory gate + one green CI run beats a fancy feature
Context bloated	`/compact`, or double-Esc rewind to the post-plan checkpoint and re-run Phase 3 tighter

Debrief Questions

1. Which toolkit artifact did the most work during the capstone - policy, skill, hook, command, or plugin?
2. Where did the grader catch something the builder believed was done?
3. What's your week-one plan: which artifact goes into your real repo Monday, and what's your CI entry point?